

Le lambda-calcul constitue un modèle de calcul universel équivalent aux machines de Turing. Un même terme peut être réduit vers sa forme normale par de multiples chemins, dont les longueurs diffèrent considérablement. Ce TIPE implémente un interpréteur de lambda-termes en OCaml, construit paresseusement le graphe orienté des réductions possibles, et applique l'algorithme A* muni d'heuristiques admissibles spécifiques aux lambda-termes pour déterminer la séquence de réduction minimale.

Table des matières

1. Introduction	3
2. Fondements théoriques	3
2.1. Syntaxe et réductions	3
2.2. Terminaison et confluence	3
2.3. Stratégies de réduction et standardisation	4
3. Implémentation de l'interpréteur	4
3.1. Représentation par indices de De Bruijn	4
3.2. Pipeline de traitement	5
4. Recherche du chemin optimal par A*	5
4.1. Formulation comme problème de plus court chemin	5
4.2. L'algorithme A* et admissibilité	5
4.3. Heuristiques admissibles pour le lambda-calcul	6
4.4. Structures de données pour la file de priorité	6
5. Conclusion	7
Bibliographie	7

1. Introduction

Le lambda-calcul, introduit par Alonzo Church dans les années 1930, formalise la notion de calcul par application et substitution de variables. Il constitue le fondement théorique des langages fonctionnels modernes tels que Haskell ou OCaml [1], [2], dans lesquels l'exécution d'un programme correspond précisément à la *béta-réduction* d'un lambda-terme.

Une propriété fondamentale de ce modèle est que la forme normale d'un terme — son résultat de calcul — est unique lorsqu'elle existe (§2.2). En revanche, les *chemins* pour l'atteindre sont multiples, et leur longueur peut varier considérablement. Par exemple, le terme $(\lambda x.(\lambda x.[I])[x]([x]xx)([x][x][x]))$ admet un chemin optimal en **15 pas**, tandis que la stratégie externe gauche — pourtant garantie de terminer — en requiert **21** et la stratégie interne gauche ne termine même pas.

Problématique. Comment appliquer la recherche de plus court chemin dans un graphe à la détermination de stratégies de réduction optimales pour les lambda-termes ?

L'objectif est triple : (1) implémenter un interpréteur supportant diverses stratégies ; (2) construire le graphe de réductions de manière paresseuse ; (3) déterminer la séquence minimale par A^* et analyser les heuristiques admissibles associées.

2. Fondements théoriques

2.1. Syntaxe et réductions

Définition 1. L'ensemble Λ des **lambda-termes** est défini inductivement par :

$$\Lambda ::= x \quad | \quad MN \quad | \quad \lambda x.M$$

où x parcourt un ensemble dénombrable de variables V , MN est l'*application* de M à N , et $\lambda x.M$ est l'*abstraction* (liaison de x dans M). Un sous-terme de la forme $(\lambda x.u), v$ est appelé un **redex**.

L'**alpha-équivalence** $\equiv_\alpha$ identifie les termes ne différant que par le renommage cohérent de variables liées : $\lambda x.x \equiv_\alpha \lambda y.y$. On travaille dans le quotient $\frac{\Lambda}{\equiv_\alpha}$, rendu décidable par l'encodage de De Bruijn (§3.1).

Définition 2. La **béta-réduction** $\rightarrow_\beta$ est la plus petite relation sur $\frac{\Lambda}{\equiv_\alpha}$ vérifiant :

$$(\lambda x.u), v \rightarrow_\beta u[x := v]$$

et stable par contexte (application et abstraction). Sa clôture réflexive-transitive est notée $\xrightarrow[\beta]^*$. Un terme sans redex est appelé **forme normale**.

La **béta-réduction ne termine pas en général** : le terme $\Omega := (\lambda x.xx)(\lambda x.xx)$ satisfait $\Omega \xrightarrow[\beta]$ Ω et n'a pas de forme normale.

2.2. Terminaison et confluence

On distingue deux niveaux de normalisation :

- **Fortement normalisant (SN)** : toutes les suites de réductions de u sont finies.
- **Faiblement normalisant (WN)** : il existe au moins une suite finie menant à une forme normale.

Théorème 1. (Confluence — Church-Rosser) Si $u \xrightarrow[\beta]{*} v_1$ et $u \xrightarrow[\beta]{*} v_2$, alors il existe w tel que $v_1 \xrightarrow[\beta]{*} w$ et $v_2 \xrightarrow[\beta]{*} w$.

Démonstration. On introduit la **réduction parallèle** $\rightarrow_{\parallel}$, qui contracte simultanément un ensemble de redex disjoints. Elle satisfait $\rightarrow_{\beta} \subseteq \rightarrow_{\parallel} \subseteq \xrightarrow[\beta]{*}$. L'étape clé est la **propriété du diamant pour $\rightarrow_{\parallel}$** : si $u \rightarrow_{\parallel} v_1$ et $u \rightarrow_{\parallel} v_2$, il existe w tel que $v_1 \rightarrow_{\parallel} w$ et $v_2 \rightarrow_{\parallel} w$. Ce w s'obtient en contractant dans v_1 (resp. v_2) les redex créés par la réduction vers v_2 (resp. v_1) ; la compatibilité des substitutions parallèles assure la cohérence. La confluence de $\xrightarrow[\beta]{*}$ s'obtient alors par induction sur la longueur des séquences [1] [ch. 3]. $\square$

Corollaire. La forme normale d'un terme, lorsqu'elle existe, est **unique à α près**. Ce corollaire justifie la notion de chemin *optimal* : tous les chemins mènent au même but unique.

2.3. Stratégies de réduction et standardisation

Une **stratégie** est une fonction sélectionnant le prochain redex à contracter. Les deux stratégies canoniques sont :

- **Stratégie interne** (appel par valeur) : on évalue l'argument avant substitution. Correspond aux langages stricts (C, OCaml par défaut).
- **Stratégie externe gauche** (ordre normal) : on réduit le redex le plus à gauche le plus à l'extérieur. C'est la seule stratégie garantie de terminer lorsqu'une forme normale existe.

Ces stratégies ne sont pas équivalentes : $(\lambda x.y), \Omega \xrightarrow[\beta]{} y$ en stratégie externe (1 pas), mais diverge en stratégie interne.

Théorème 2. (Standardisation) Si u est faiblement normalisant, la stratégie externe gauche calcule la forme normale de u en un nombre fini de pas.

Démonstration (esquisse). Une **réduction standard** est une séquence où aucun redex n'est contracté à gauche d'un redex précédemment contracté. Le théorème de standardisation (Curry, 1958) affirme que toute réduction peut être réordonnée en réduction standard [2]. La stratégie externe gauche produit précisément de telles séquences. Si le terme est WN, la réduction standard doit atteindre la forme normale : une réduction standard bloquée sur un terme non-normal contiendrait un redex contractable à droite, contradiction. La terminaison découle d'un ordre bien fondé sur la position du redex contracté à chaque étape. $\square$

Conséquence. Le théorème de standardisation garantit que notre programme termine pour tout terme WN, même si A^* ne trouve pas le chemin optimal dans le temps imparti.

3. Implémentation de l'interpréteur

3.1. Représentation par indices de De Bruijn

La gestion de l'alpha-équivalence est cruciale : sans elle, des termes identiques seraient traités comme distincts dans le graphe de réductions, causant des explosions combinatoires. Les **indices de De Bruijn** remplacent chaque variable liée par sa *profondeur de liaison* :

$$\lambda x. \lambda y. x, y \longrightarrow \lambda. \lambda. 1 \ 0$$

Avec cet encodage, l'alpha-équivalence devient l'**égalité syntaxique**, ce qui permet une comparaison de termes en $O(n \cdot v)$ (où n est la taille de l'arbre et v la profondeur maximale des variables liées) — décisive pour détecter les sommets déjà visités dans le graphe.

3.2. Pipeline de traitement

Le programme transforme une chaîne de caractères en chemin optimal par quatre étapes :

Étape	Transformation	Note de complexité
1	Chaîne $\rightarrow$ Tokens	linéaire
2	Tokens $\rightarrow$ AST	linéaire
3	AST $\rightarrow$ Graphe (paresseux)	$O(n^3)$ par sommet
4	Graphe $\rightarrow$ Plus court chemin	A*, guidé par h

Tableau 1. – Pipeline de traitement

La construction du graphe (étape 3) est **paresseuse** : les successeurs d'un nœud ne sont calculés que lors de son extraction de la file de priorité. Cette approche est indispensable car le graphe peut être infini (terme divergent) ou d'une taille rédhibitoire ($[x][x]$: 310 000 arêtes).

Les complexités des opérations élémentaires (avec n = nombre de nœuds, m = taille du terme substitué) sont :

Opération	Complexité
Égalité structurelle (De Bruijn)	$O(n \cdot v)$
Substitution de x par v dans u	$O(n + m^2)$
Étape de réduction complète	$O(n^3)$

Tableau 2. – Complexités des opérations critiques

4. Recherche du chemin optimal par A*

4.1. Formulation comme problème de plus court chemin

Le **graphe de réductions** G_u d'un terme u est le graphe orienté dont les sommets sont les termes accessibles depuis u par $\xrightarrow[\beta]{*}$, et les arêtes sont les bêta-réductions élémentaires (chacune de poids 1). La forme normale de u , si elle existe, est l'unique puits de ce graphe (par confluence). Le problème est donc : trouver le chemin de poids minimal de u vers sa forme normale dans G_u .

4.2. L'algorithme A* et admissibilité

Théorème 3. (Optimalité de A*) L'algorithme A* retourne un plus court chemin de la source vers le but si et seulement si l'heuristique h est **admissible** : $\forall u, h(u) \leq d^*(u)$, où $d^*(u)$ est le vrai nombre minimum de réductions pour normaliser u .

A* sélectionne à chaque étape le nœud minimisant $f(u) = g(u) + h(u)$, où $g(u)$ est le nombre de réductions déjà effectuées depuis le terme initial. L'exploration est **paresseuse** : les successeurs ne sont générés que lorsqu'un nœud est extrait. Cela permet d'opérer sur un graphe effectivement infini, sous réserve que A* converge.

La supériorité de A* sur Dijkstra est illustrée par le terme $(\lambda x.[x], x, x)([x], [x], [x])$:

Méthode	Pas de réduction	Optimalité garantie
Stratégie interne	> 19	Non
Stratégie externe gauche	19	Non
A* (heuristique h_2)	13	Oui

Tableau 3. – Comparaison sur $(\lambda x.[x]xx)([x][x][x])$

(Les graphes de réduction correspondants seront fournis en illustration.)

4.3. Heuristiques admissibles pour le lambda-calcul

Trois heuristiques ont été développées et étudiées [3] :

h_1 — Taille du terme. $h_1(u)$ = nombre de nœuds de l’AST de u . Intuitivement, chaque réduction tend à simplifier le terme, mais une substitution peut dupliquer des sous-termes et *augmenter* la taille. h_1 peut donc sur-estimer $d^*(u)$ et n’est **pas admissible** en général.

h_2 — Nombre de redex. $h_2(u)$ = nombre de redex dans u . Chaque pas contracte un redex, mais peut en créer de nouveaux. L’admissibilité requiert que tout terme à k redex soit normalisable en au moins k pas — ce qui n’est pas garanti si chaque réduction peut créer plusieurs redex supplémentaires.

h_3 — Nombre de spine redex. La *stratégie de la colonne vertébrale* de Barendregt [3] réduit en priorité les redex sur le chemin principal (spine) du terme. $h_3(u)$ = nombre de spine redex constitue une heuristique plus informée. Son admissibilité découle de la théorie des *réductions nécessaires* : tout chemin vers la forme normale doit contracter au moins une fois chaque spine redex.

Heuristique	Admissible	Remarque
h_1 : taille	Non	Peut sur-estimer par duplication
h_2 : # redex	À vérifier	Dépend du terme
h_3 : # spine redex	Oui (conjecture)	Lié aux réductions nécessaires
$h = 0$ (Dijkstra)	Oui	Triviale, non informative

Tableau 4. – Admissibilité des heuristiques étudiées

4.4. Structures de données pour la file de priorité

L’efficacité de A* repose sur celle de la file de priorité (extraction du minimum et insertion). On compare empiriquement plusieurs structures sur deux termes de référence :

Structure	$[x][x]$	$[x][x]$	$[x][x]$
Tas binaire	0,012 s	3 min 44 s	—
Tas binomial	0,012 s	3 min 25 s	—
Tas 2-3	—	—	—
Tas de Fibonacci	—	—	—
Pairing heap	—	—	—

Tableau 5. – Temps d’exécution selon la structure de file de priorité (résultats partiels)

Les complexités théoriques divergent : le tas de Fibonacci offre $O(1)$ amorti en insertion et $O(\log n)$ en extraction, contre $O(\log n)$ pour le tas binaire dans les deux cas [4]. L’avantage

pratique dépend toutefois de la localité mémoire et des constantes cachées, d'où la nécessité du benchmark expérimental.

5. Conclusion

Ce TIPE a formalisé la recherche de stratégie de réduction optimale comme un problème de plus court chemin dans un graphe potentiellement infini, et a conduit au développement d'un interpréteur OCaml complet mettant en œuvre A^* avec des heuristiques spécifiques au lambda-calcul. Les résultats préliminaires confirment un gain significatif (13 pas contre 19 sur l'exemple présenté), avec des perspectives d'amélioration par le choix de la file de priorité et l'affinement des heuristiques.

Les questions ouvertes portent sur la caractérisation précise des termes pour lesquels h_2 et h_3 sont admissibles, le lien avec la théorie des **réductions optimales** de Lévy [5], et l'éventuelle parallélisation de l'exploration.

Bibliographie

- [1] H.P. Barendregt, *The lambda Calculus - Its Syntax and Semantics*.
- [2] Jean Goubault-Larrecq, « Lambda-calcul et langages fonctionnels ». [En ligne]. Disponible sur: <https://lsv.ens-paris-saclay.fr/~goubault/Lambda/lambda.pdf>
- [3] H.P. Barendregt, « Needed Reduction and Spine Strategies for the Lambda Calculus ». [En ligne]. Disponible sur: <https://www.sciencedirect.com/science/article/pii/0890540187900010?via%3Dihub>
- [4] « D-ary heap - Wikipedia ». [En ligne]. Disponible sur: https://en.wikipedia.org/wiki/D-ary_heap
- [5] Jean-Jacques Lévy, « Optimal Reductions in the lambda-calculus ».